15/10/2026

Ant Colony optimization

Code:

```
import random
import matplotlib.pyplot as plt
import networkx as nx

# 1. Define the network and link costs
network = {
    'A': {'B': 2, 'C': 4},
    'B': {'A': 2, 'C': 1, 'D': 3, 'E': 5},
    'C': {'A': 4, 'B': 1, 'D': 2},
    'D': {'B': 3, 'C': 2, 'E': 1, 'F': 4},
    'E': {'B': 5, 'D': 1, 'F': 2},
    'F': {'D': 4, 'E': 2}
}

source = 'A'
destination = 'F'

energy = {
    'A': 100, 'B': 80, 'C': 90,
    'D': 70, 'E': 85, 'F': 100
}
```

```
# 2. ACO parameters
num_ants = 10
iterations = 50
alpha = 1.0
beta = 5.0
rho = 0.5

random.seed(42)

# 3. Initialize pheromones
pheromone = {}

for node in network:
    for neighbor in network[node]:
        pheromone[(node, neighbor)] = 1.0



# 4. Select the next node
def choose_next(current, visited):
    candidates = []
    weights = []

    for neighbor, cost in network[current].items():
        if neighbor not in visited:
            heuristic = (energy[neighbor] / 100) / cost

            weight = (
                pheromone[(current, neighbor)] ** alpha
                * heuristic ** beta
```

```
            )

            candidates.append(neighbor)
            weights.append(weight)

    if not candidates:
        return None

    return random.choices(candidates, weights=weights, k=1)[0]



# 5. Construct a route from source to destination
def construct_route():
    current = source
    route = [source]
    visited = {source}
    total_cost = 0

    while current != destination:
        next_node = choose_next(current, visited)

        if next_node is None:
            return None, float('inf')

        total_cost += network[current][next_node]
        current = next_node
        route.append(current)
        visited.add(current)
```

```
    return route, total_cost



# 6. Run the ACO algorithm
best_route = None
best_cost = float('inf')
cost_history = []

for iteration in range(iterations):
    routes = []

    for ant in range(num_ants):
        route, cost = construct_route()

        if route is not None:
            routes.append((route, cost))

            if cost < best_cost:
                best_cost = cost
                best_route = route

    # Pheromone evaporation
    for edge in pheromone:
        pheromone[edge] *= (1 - rho)

    # Pheromone deposit
    for route, cost in routes:
        deposit = 1 / cost
```

```
        for i in range(len(route) - 1):
            u = route[i]
            v = route[i + 1]

            pheromone[(u, v)] += deposit
            pheromone[(v, u)] += deposit

    cost_history.append(best_cost)



# 7. Print the result
print("Best Route:", " -> ".join(best_route))
print("Total Route Cost:", best_cost)
print("Number of Iterations:", iterations)



# 8. Create the network graph
G = nx.Graph()

for node in network:
    G.add_node(node)

for node in network:
    for neighbor, cost in network[node].items():
        if not G.has_edge(node, neighbor):
            G.add_edge(node, neighbor, weight=cost)

# Fixed positions for a clear network diagram
pos = {
```

```
    'A': (0, 1),
    'B': (1, 2),
    'C': (1, 0),
    'D': (2.5, 2),
    'E': (2.5, 0),
    'F': (3.5, 1)
}

# Highlight the best route
route_edges = list(zip(best_route[:-1], best_route[1:]))

node_colors = []
for node in G.nodes():
    if node == source:
        node_colors.append('lightgreen')
    elif node == destination:
        node_colors.append('salmon')
    else:
        node_colors.append('skyblue')

# Display node energy as part of each label
node_labels = {
    node: f"{node}\nEnergy: {energy[node]}"
    for node in G.nodes()
}

fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Left: Network topology and best route
```

```
ax = axes[0]

nx.draw_networkx_edges(
    G, pos, ax=ax,
    edge_color='gray', width=1.5
)

nx.draw_networkx_edges(
    G, pos, ax=ax,
    edgelist=route_edges,
    edge_color='green', width=4
)

nx.draw_networkx_nodes(
    G, pos, ax=ax,
    node_color=node_colors,
    node_size=2200,
    edgecolors='black'
)

nx.draw_networkx_labels(
    G, pos, labels=node_labels,
    ax=ax, font_size=9
)

edge_labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(
    G, pos, edge_labels=edge_labels,
    ax=ax, font_size=10
```

```
)

ax.set_title("ACO Network Routing\nGreen = Best Route")
ax.axis('off')

# Right: Convergence graph
ax = axes[1]

ax.plot(
    range(1, iterations + 1),
    cost_history,
    marker='o',
    markersize=3,
    color='blue',
    label='Best route cost'
)

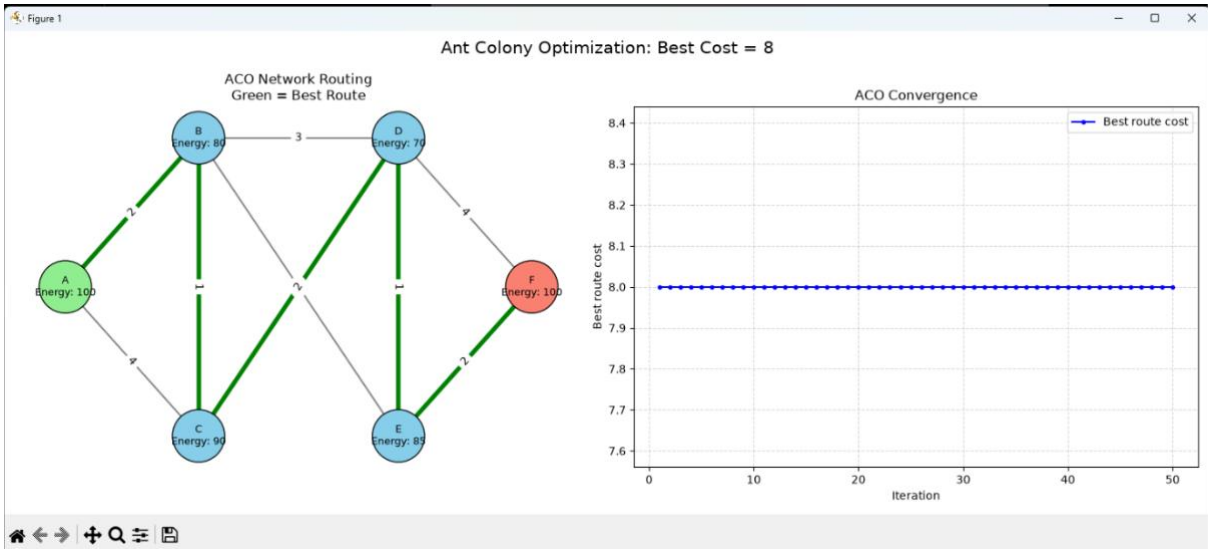
ax.set_title("ACO Convergence")
ax.set_xlabel("Iteration")
ax.set_ylabel("Best route cost")
ax.grid(True, linestyle='--', alpha=0.5)
ax.legend()

plt.suptitle(
    f"Ant Colony Optimization: Best Cost = {best_cost}",
    fontsize=15
)

plt.tight_layout()
```

plt.show()

output:



Nischal ar

1wn24cs182